Configuration and Reproducibility Reporting in Public Robot Manipulation Simulation Benchmarks

Review

Xiongyule Zhou

School of Mechanical, Electronic and Control Engineering, Beijing Jiaotong University, Beijing 100044, China

Correspondence: jensenzhou47@gmail.com

Abstract

Public simulation benchmarks support robot manipulation research, but a benchmark name rarely identifies the configuration behind a reported result. This critical review examines how publications and public artifacts expose the conditions needed to reconstruct manipulation experiments. Ten benchmark or framework entries are compared across software identity, robot models, control interfaces, randomness, and task and evaluation configuration. The evidence comprises published descriptions, official documentation, and source files retrieved on 19 October 2026, with repository observations anchored to immutable commits. The comparison distinguishes available configuration mechanisms from the settings selected for a particular experiment and from evidence obtained by executing that experiment. Concrete examples include simulator dependency ranges, controller composition, per-environment seeds, serialized initial states, and evaluation horizon changes. These examples show why reporting isolated fields can leave dependencies unresolved even when substantial implementation information is public. A dependency-based framework connects each reported claim to its software, embodiment, action, reset, and evaluation conditions. An accompanying evidence table and reporting manifest make those connections inspectable. No policies were retrained, and no simulator performance or reproducibility rankings are inferred. The review provides a practical basis for evaluating configuration traceability before committing resources to experimental reproduction.

Keywords: robot manipulation; simulation benchmarks; reproducibility; configuration management; control interfaces; random seeds; evaluation protocols; research software

1 Introduction

Robot manipulation benchmarks make learning methods easier to compare by providing reusable environments, tasks, and evaluation procedures. Yet an experiment is specified by more than a benchmark name. Simulator releases, robot assets, controller implementations, reset distributions, and success criteria jointly determine the problem presented to an agent. Recovering those choices is a prerequisite for interpreting a result, even when reproducing the complete training process is impractical.

Reinforcement learning research has already established that implementation details and experimental design can materially affect conclusions [1,2]. Statistical evaluation adds another requirement: an aggregate score should represent the specified population of runs and tasks, with uncertainty appropriate to that population [3,4]. These concerns address different sources of uncertainty. Additional random seeds cannot resolve an unknown controller, and a fully specified controller cannot resolve selective checkpoint evaluation.

Recent reviews provide useful maps of the surrounding literature. Wong et al. compare physics simulators for embodied navigation and manipulation, including their capabilities and hardware demands [5]. Acs and Zhong map long-horizon manipulation methods and explicitly discuss evaluation and reporting inconsistencies [6]. Muratore et al. organize domain randomization methods and their role in simulation-to-reality transfer [7]. Their contributions establish that simulator choice, evaluation design, and randomization matter. They do not remove the practical need to locate a configuration in the artifacts attached to a particular benchmark release.

This review addresses that narrower problem. Its unit of comparison is a benchmark or framework entry together with its publication and accessible implementation, rather than an individual learning algorithm. The questions are: where can the essential configuration information be located; which dependencies prevent apparently similar configurations from being equivalent; and what should be reported before a reproduction attempt begins? The scope includes manipulation environments used for reinforcement learning, imitation learning, and language-conditioned evaluation. It excludes navigation-only simulators, physical robot replication protocols, and comprehensive comparisons of policy performance.

The contribution is an evidence-linked synthesis of configuration reporting across ten entries, supported by a reusable extraction table. A five-dimension framework separates software identity, embodiment, control, randomness, and task evaluation while explicitly connecting their dependencies. A reporting manifest then translates the synthesis into a practical artifact for authors and reviewers. These contributions complement existing checklists and reviews; they do not establish a new definition of computational reproducibility or claim an exhaustive census of robotics software.

2 Review Design and Evidence Boundaries

2.1 Review type and case selection

This is a structured critical review with a purposive comparative sample. It is neither a systematic review nor a PRISMA scoping review. Search activity, source retrieval, and extraction were conducted on 19 October 2026. The search combined terms for robot manipulation, simulation, benchmarks, reproducibility, configuration, software versions, controllers, seeds, and evaluation. General web discovery was followed by retrieval from primary publication pages, author manuscripts, official project documentation, and project repositories. The exact discovery queries and access outcomes are retained in the supplementary search record.

The sample was chosen to expose different configuration dependencies. robosuite, RLBench, and Meta-World represent established reusable manipulation environments [8,9,10]. ManiSkill2 and ManiSkill3 provide a related pair spanning substantial simulator and execution changes [11,12]. CALVIN and LIBERO provide language-conditioned and structured task evaluation [13,14]. RoboCasa represents scene-rich household manipulation, with its current repository also documenting the RoboCasa365 release [15,16]. robomimic represents dataset-mediated environment reconstruction, while Ravens exposes primitive-level manipulation and dataset-linked evaluation [17,18].

Eligibility required manipulation relevance, an accessible technical publication or author manuscript, and an official public implementation with inspectable configuration mechanisms. Selection favored complementary mechanisms rather than citation rank or benchmark size. ManiSkill2 and ManiSkill3 are separate entries because their releases expose meaningfully different configuration conditions. They are not independent observations of community practice. Shared dependencies also connect robosuite, LIBERO, RoboCasa, and robomimic. No inclusion percentage, field-wide prevalence estimate, or statistical inference is calculated from this sample.

2.2 Source retrieval and version control

Publications were read for benchmark purpose, action and observation design, and evaluation context. Repository files were inspected for installation declarations, robot descriptions, controller construction, randomization, task generation, and evaluation entry points. Source selection was adaptive: documentation references and imports were followed where needed to interpret a selected field. This was not an exhaustive inspection of every file, task, dataset, or historical release.

Repository observations were anchored to the default-branch commit returned during retrieval. One explicit exception was ManiSkill2, whose historical v0.5.3 tag was resolved in the unified ManiSkill repository. The separate ManiSkill2 repository had become a migration entry point. For CALVIN, the parent tree was inspected to identify the pinned calvin_env submodule commit, and selected configuration files were retrieved at that commit. Table S1 contains stable file links and line locators; Table S2 identifies the repository snapshots. A source manifest records access dates, URLs, response outcomes, and SHA-256 checksums of downloaded material. A source index accompanies the supplementary tables; third-party full texts are retained separately for checking.

The publication and repository dates are treated separately throughout the review. A current package declaration is evidence about the retrieved source, not evidence about the environment used in an older paper. Mutable documentation pages were retained as dated snapshots and cross-checked against pinned source when they informed case-specific observations. Failed access attempts and mismatched candidate records were retained in the research log rather than silently counted as read publications.

2.3 Extraction and synthesis

Each case was examined across five dimensions: software identity, robot model, control interface, randomness, and task and evaluation configuration. Fifty source observations cover these dimensions, with four additional observations documenting compatibility, timing, or sequence generation. Each row states what was observed, its

location, and the inference that the source cannot establish. These rows are literature-derived descriptions of public artifacts, not newly measured simulator outcomes.

Extraction and checking were conducted in one review workflow. No independent duplicate selection or extraction was performed, and no inter-rater agreement was estimated. There was no author correspondence, installation testing, or policy rerun. Bibliographic identity was checked against primary publication records, arXiv metadata, and exact digital object identifier (DOI) records where available. Ambiguous publication variants were kept distinct rather than combining one version's title or author list with another version's metadata. The analysis uses explicit positive evidence and bounded inferences; failure to inspect a field is not coded as proof that it is absent from a project.

The synthesis follows configuration dependencies instead of assigning a summed completeness score. Five named fields can still leave an experiment underspecified when they do not resolve to the same run. Conversely, an apparently brief paper may point to a precise configuration stored in an artifact. The proposed framework therefore concerns traceability to an experimental condition, while the comparison tables describe where supporting information can be found.

Table 1. Relationship to adjacent reviews and reproducibility guidance.

Source	Main contribution relevant here	Additional question addressed in this review
Wong et al. [5]	Physics-simulator capabilities, manipulation and navigation support, hardware considerations	Which source revision and resolved configuration define a specific manipulation evaluation?
Acs and Zhong [6]	Long-horizon reinforcement learning methods, evaluation landscape and reporting recommendations	Where can a benchmark's reported conditions be located in its implementation?
Muratore et al. [7]	Domain randomization methods and transfer considerations	How are randomization distributions, generators and initial states tied to a particular run?
Lynnerup et al. [2]	Reproducibility and evaluation of reinforcement learning on real robots	Which simulation-specific asset, wrapper and timing dependencies need inspection?
Pineau et al. and ACM SIGSIM guidance [19,20]	Artifact dissemination, evaluation and reproducibility practices	How can these distinctions organize source evidence without claiming execution-based reproduction?

The comparison identifies complementary scope. It does not assert that adjacent sources omit every implementation detail or that this review is the first reporting checklist.

3 A Dependency-Based Framework for Configuration Reporting

3.1 Three levels of evidence

Reproducibility programs distinguish disseminating artifacts from evaluating them and from reproducing results [19,20]. This review applies that distinction to manipulation simulation through three evidence levels. Publication evidence describes the intended method or benchmark condition. Artifact evidence identifies an implementation, configuration file, asset, or evaluation procedure. Execution evidence records what happened when a specified artifact was run under specified conditions.

These levels answer different questions. A publication may identify an operational-space controller without recording its resolved gains. A repository may provide those gains without identifying the experiment that selected them. An execution log may record the chosen configuration without establishing that an independent group obtained the same scientific conclusion. None of these statements invalidates the others; their evidential roles differ.

All case-specific observations in Table S1 are artifact evidence interpreted alongside publications. They establish inspectable configuration mechanisms and selected declared values. They do not establish successful installation, deterministic replay, reproduced success rates, or real-world transfer. This boundary avoids turning documentation inspection into an unsupported audit of experimental outcomes.

3.2 Five configuration dimensions

Software identity includes the framework commit, simulator build, language runtime, dependencies, rendering backend, and platform constraints. A release name is useful but can be insufficient when editable installations or

externally downloaded assets introduce additional revisions. Direct package pins narrow the installation space; a resolved environment record identifies a particular installation more precisely.

Embodiment includes the robot description, kinematic structure, collision geometry, inertial parameters, actuator limits, gripper, and sensor attachments. A robot family name identifies only part of this condition. A Panda model loaded from one asset tree need not be interchangeable with a modified Panda model that changes finger geometry or controller settings. Whether these differences matter depends on the claim being evaluated.

The control interface translates the policy output into simulator behavior. Its specification includes action units, coordinate frame, absolute or incremental targets, normalization, controller gains, interpolation, control frequency, and physics substeps. Primitive-level commands require the primitive implementation and stopping conditions. These requirements apply even when two interfaces return action vectors with the same dimension.

Randomness includes training initialization, exploration, task sampling, environment reset, asset selection, demonstration selection, and evaluation. A reproducible random sequence requires an identified generator and a defined relationship between its state and the sampled quantities. Parallel environments introduce additional questions about seed assignment and reset ordering. Saved environment states can constrain initial conditions but need compatible model and asset definitions.

Task and evaluation configuration includes task versions, object and scene splits, observations, rewards, horizon, termination, success predicates, checkpoint selection, episode aggregation, and uncertainty reporting. Training and evaluation may deliberately use different conditions. Those differences should be explicit because they define what generalization is being measured. Table 2 lists the minimum questions associated with each dimension.

Table 2. Five configuration dimensions and their dependency questions.

Dimension	Minimum reconstruction question	Dependency requiring explicit linkage
Software identity	Which source, simulator build and resolved installation were used?	Framework, imported packages, binaries, renderer and assets
Robot model	Which robot, gripper and sensor descriptions were instantiated?	Asset revisions, physical parameters and controller assumptions
Control interface	What does one policy action cause the environment to execute?	Frames, scaling, gains, wrappers, frequencies and primitive duration
Randomness	Which generator or saved state determines each stochastic choice?	Run/episode mapping, parallel reset order, task generator and asset set
Task and evaluation	Which population and procedure produced the reported quantity?	Task split, horizon, success predicate, checkpoint and aggregation

This is a synthesis framework, not a validated measurement scale. The necessary detail depends on the scientific claim.

3.3 Why dependencies matter more than field counts

Figure 1 represents configuration reporting as a dependency graph. The claim being assessed sits above a run configuration that resolves into interacting components. A controller depends on robot state and timing; a reset seed depends on the task generator and assets; an evaluation score depends on success predicates, horizon, and aggregation. A missing connection can be more consequential than several well-documented peripheral fields.

For example, reporting a seed and task name does not identify an initial scene when the object sampling distribution changes between releases. Reporting a simulator version and robot model does not identify the action semantics when a wrapper applies normalization or replaces a low-level action with a motion primitive. These are logical consequences of how the experiment is constructed, not measured effect-size claims about the selected projects.

The required resolution is claim-relative. Reconstructing a trained policy's evaluation requires its checkpoint, observation preprocessing, environment condition, and evaluation procedure. Reproducing a training comparison additionally requires datasets, optimizer settings, training budgets, run allocation, and model selection. A physics accuracy claim would require different evidence, including reference measurements and calibration conditions. Configuration traceability is therefore a prerequisite for several reproduction goals, rather than a sufficient guarantee for any of them.

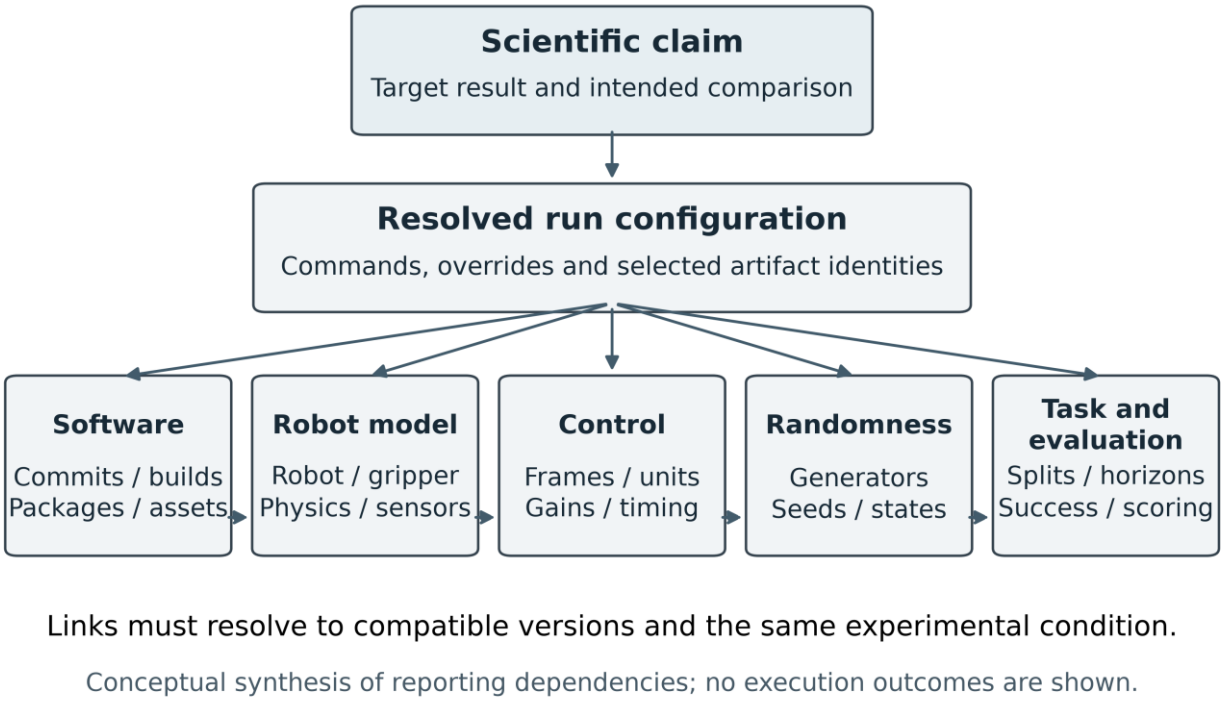


Figure 1. Dependency-based framework for configuration reporting. A claim is associated with a resolved run configuration whose software, model, control, randomness, and evaluation conditions must be compatible. Arrows indicate reporting relationships, not measured causal effects or experimental data.

4 Comparative Evidence from Public Benchmarks

4.1 Software identity and release drift

The retrieved source files illustrate several ways to specify software identity. RLBench's README names CoppeliaSim v4.1.0 and PyRep [21]. LIBERO's requirements explicitly pin robosuite 1.4.0 and robomimic 0.2.0 [22]. The historical ManiSkill2 v0.5.3 requirements pin SAPIEN 2.2.2 [23]. These are useful declarations, although none alone describes every transitive dependency or downloaded binary.

Other declarations preserve more installation flexibility. The robosuite snapshot declares version 1.5.2 and a MuJoCo dependency interval of at least 3.3.0 and below 3.10 [24]. ManiSkill3 uses platform-dependent SAPIEN minimum versions, including 3.0.3 on Linux [25]. Ravens combines exact pins for some packages with a PyBullet lower bound [26]. Table 3 compares these declarations without interpreting tighter constraints as evidence of better experimental reproducibility.

Meta-World provides a particularly clear temporal distinction. Its retrieved package metadata declares version 3.1.1 and MuJoCo 3.3.0; its README uses version-3 task identifiers [27]. These observations concern the current implementation, while the original benchmark publication describes an earlier experimental context. A comparison citing only the publication and installing the current branch leaves the relationship between those contexts unresolved.

Current RoboCasa documentation makes release-specific evaluation changes explicit. The README identifies RoboCasa365 as the latest iteration and records a v1.0.1 update on 12 May 2026 that increased task horizons by a factor of 1.5 [28]. This supports a concrete reporting requirement: release identity and horizon must accompany a benchmark result. It does not show how much any policy's score changed, and this review did not measure that effect.

CALVIN adds a dependency through version-control structure. Its installation instructions request recursive submodule checkout, and the parent source tree fixes a calvin_env commit [29]. Recording the parent revision while independently updating the environment submodule produces a different source condition. The comparison therefore includes the submodule identity rather than treating a repository URL as a complete software specification.

Table 3. Software declarations found at the inspected source snapshots.

Entry and source	Inspected identity or dependency declaration	Condition that still needs to be resolved
robosuite [24]	Package 1.5.2; MuJoCo >=3.3.0,<3.10	Installed simulator version and run-specific options
RLBench [21]	README names CoppeliaSim v4.1.0 and PyRep	PyRep commit, binary provenance and selected action mode
Meta-World [27]	Package 3.1.1; MuJoCo 3.3.0; current task identifiers use v3	Relationship to earlier benchmark releases and experimental splits
ManiSkill2 [23]	Historical tag v0.5.3; SAPIEN 2.2.2	Other resolved dependencies, assets and selected modes
ManiSkill3 [25]	Platform-specific SAPIEN minimum versions; Linux >=3.0.3	Installed backend, platform and parallel configuration
CALVIN [29]	Parent commit plus pinned calvin_env submodule	Submodule checkout, composed configuration and dataset release
LIBERO [22]	robosuite 1.4.0 and robomimic 0.2.0 pinned	Resolved binary dependencies and task/initial-state artifacts
RoboCasa [28]	Current package 1.0.1; MuJoCo 3.3.1; README tracks RoboCasa365	Original or current release, robosuite revision and horizon protocol
robomimic [30]	Installation dependencies and environment-wrapper utilities	Dataset/checkpoint environment metadata and compatible simulator
Ravens [26]	Mixed exact pins and ranges, including PyBullet >=3.0.4	Resolved installation and execution entry point

Observed on 19 October 2026. Declarations describe the cited snapshot, not necessarily an original paper's runtime. Direct dependency pins are not complete environment locks. Full commits are listed in Table S2; evidence rows use suffix V in Table S1.

4.2 Robot models and control interfaces

The selected artifacts expose robot information at different levels. robosuite provides robot classes and controller configuration files; ManiSkill2 identifies a Panda model in Unified Robot Description Format (URDF) together with material and controller parameters [24,23]. CALVIN's pinned environment configuration names a Panda description and action-related scaling settings [29]. RLBench exposes a robot_setup option, while LIBERO's environment wrapper defaults to Panda [21,22]. These mechanisms make the embodiment inspectable, but the selected configuration still needs to be associated with the evaluated policy.

Controller names also have implementation-specific meaning. The inspected robosuite OSC_POSE configuration specifies incremental inputs, a base reference frame, output bounds, gains, and damping. LIBERO selects OSC_POSE through its environment wrapper and relies on its pinned robosuite version. A reader should therefore resolve the controller in that dependency rather than assume that the current robosuite configuration defines LIBERO's historical action interface [24,22].

RLBench's MoveArmThenGripper action mode executes the arm component before the gripper component. Its arm mode can determine whether an action invokes planning or another control mechanism [21]. Ravens dispatches manipulation through action primitives, whereas Meta-World documents translation and gripper action components [26,27]. A primitive execution and a low-level action update are different units of interaction. Equal episode lengths measured in environment steps consequently need not represent equal control opportunities.

Timing can also be resolved incorrectly by reading a default in isolation. Ravens' environment constructor defaults to 240 Hz, while its demonstration-generation entry point passes 480 Hz. The simulator timestep is set from the supplied frequency [26]. The effective condition therefore follows the executed entry point and its overrides. The example does not establish a defect; it establishes why source-level defaults should not be substituted for a run's resolved arguments.

ManiSkill2 separately declares simulation and control frequencies in its base environment, with defaults of 500 Hz and 20 Hz [23]. Those defaults illustrate how timing can be exposed clearly, while also showing why task-specific overrides remain relevant. For policies involving contact, controller and timing information should be reported together. This recommendation concerns specification completeness, without asserting an unmeasured relationship between frequency and task success.

4.3 Random seeds and reconstructing initial conditions

The artifacts distinguish several objects often compressed into the word seed. robomimic's training script initializes NumPy and PyTorch from a configuration seed [30]. RLBench's live demonstration generation stores NumPy state before resetting the scene, and its reset_to_demo procedure restores a demonstration's variation [21]. CALVIN's evaluator seeds workers and uses a separate multistep sequence generator [29]. These mechanisms constrain different parts of an experiment and cannot be treated as interchangeable seed reports.

ManiSkill3's randomization documentation explicitly discusses a seed for each parallel environment and supports a list passed to reset. It also distinguishes randomization during environment loading from recovery of an environment state [25]. This identifies the intended random-number mechanism more precisely than a generic statement that an experiment is seeded. It remains narrower than a demonstration of identical physics trajectories across different hardware or simulator builds.

LIBERO's evaluator loads serialized initial states and applies them to the environment. Ravens separates training and testing seed parity during demonstration generation and retrieves episode seeds from its test dataset [22,26]. These approaches make the reset condition partly data-dependent. A seed list without the relevant initial-state file, task identity, or dataset episode indices may therefore leave the intended evaluation distribution ambiguous.

Simulator state also has a software context. MuJoCo documentation describes simulation state and the conditions relevant to deterministic simulation [31]. A stored state is interpreted by a model and simulator implementation; it is not a universal scene description. For a state-based reset protocol, the reporting object should therefore include the state representation, asset identity, compatible software, and the order of any subsequent settling actions.

The resulting comparison in Table 4 avoids a binary seeded or unseeded judgment. It identifies the mechanism visible in the inspected source and the condition that mechanism does not independently establish. This distinction also protects against a common overinterpretation: deterministic generation of random numbers is not equivalent to statistical reproducibility of a learning result.

Table 4. Control and reset mechanisms and the limits of the resulting evidence.

Entry	Mechanism visible in the inspected source	Additional condition for interpreting a result
robosuite	Controller JSON specifies delta pose semantics; base environment accepts a seed	Selected controller, resolved parameters, timing and all relevant RNG streams
RLBench	Arm-then-gripper composition; demonstration generation stores NumPy state	Selected arm/planner mode, variation and demonstration index
Meta-World	Translation/gripper action documentation; seeding of environment and spaces	Task version and split, action scaling and evaluated goal instances
ManiSkill2	Separate physics/control rates; main and episode RNG mechanisms	Task overrides, reconfiguration and selected control/observation modes
ManiSkill3	Per-environment seed lists; randomization and state recovery distinguished	Seed-to-environment mapping, backend, assets and saved state format
CALVIN	Worker seeding; separate task-sequence generation and initial-condition utilities	Training seeds, sequence set, composed environment and checkpoint
LIBERO	OSC_POSE wrapper; evaluator loads and applies initial states	Pinned controller dependency, initial-state file and selected row indices
RoboCasa	Composite controller factory; seed, asset and camera/layout sampling options	Embodiment, distribution parameters, release and evaluator horizon
robomimic	NumPy/PyTorch training seeds; environment metadata read from HDF5	Selected metadata values, dataset filtering, checkpoint and evaluation resets
Ravens	Primitive execution; training/test seed parity and dataset-stored episode seeds	Primitive implementation, seed schedule, task mode and episode list

RNG, random-number generator; JSON, JavaScript Object Notation; HDF5, Hierarchical Data Format version 5. Source references match Table 3. Table S1 provides individual robot (M), control (C), randomness (R) and task (T) evidence locators. None of these mechanisms was executed in this review.

4.4 Task configuration and the evaluation contract

Task identity includes more than a readable task name. Meta-World distinguishes benchmark splits and documents evaluation through success rate rather than episodic training reward [27]. LIBERO combines task descriptions, suite membership, and initial-state files. CALVIN combines a task-sequence generator with evaluator constants: the inspected evaluator sets 1000 sequences and a per-subtask rollout limit of 360 steps [22,29]. These are descriptions of source behavior, not newly generated evaluation results.

Scene-rich benchmarks add another dependency between task identity and the sampled environment. RoboCasa's environment factory exposes layout, style, object-instance, and camera-randomization settings together with the seed [28]. A comparison using the same nominal task can still concern different generalization conditions if those settings select different asset populations. Reporting the task distribution is therefore necessary alongside reporting a task label.

robomimic illustrates how configuration can be carried by datasets and checkpoints. Its utilities read environment metadata from an env_args attribute in Hierarchical Data Format version 5 (HDF5) files, and the training workflow writes its resolved configuration to a JavaScript Object Notation (JSON) file [30]. These are mechanisms for retaining provenance. This review did not inspect every released dataset or checkpoint, so it does not claim that all such files contain complete or mutually compatible metadata.

Compatibility settings can reveal relationships that package versions alone conceal. robosuite documents a lite_physics option intended to preserve backward compatibility with datasets collected using versions up to 1.4.1 [24]. This is evidence that replay conditions may depend on an environment option as well as a version label. It does not demonstrate that setting the option reproduces every historical trajectory.

Evaluation should be specified as a contract connecting input conditions to a reported quantity. That contract includes the checkpoint-selection rule, observation preprocessing, task and reset populations, horizon, failure handling, success predicate, and aggregation. Gymnasium's API distinction between termination and truncation illustrates why an interface field can affect how an episode is interpreted [32]. Statistical recommendations then apply to the defined population of runs and tasks, rather than correcting an otherwise unspecified evaluation [3,4].

5 A Practical Reporting Manifest

5.1 Report resolved choices alongside available options

The comparison suggests a practical reporting rule: provide a resolved configuration for the claim being made, while linking it to the source that implements each consequential choice. A framework may support many robots, controllers, and observation modes. Listing those capabilities does not reveal which combination generated a particular table or figure. Table 5 translates the five dimensions into a compact set of reporting objects.

The supplementary manifest template organizes these objects under claim, software, embodiment, control, randomness, task, evaluation, and provenance. It uses null values for information that a future author must supply. It is a reporting template, not an executable benchmark configuration or a validated standard. The separate provenance section links a result identifier to a command, resolved configuration, dataset, checkpoint, and raw evaluation output.

For software, both source identity and the resolved installation matter. A repository commit supports inspection of implementation behavior, while package and binary records describe the environment in which that behavior was executed. Downloaded assets should have their own identifiers and checksums when they can change independently. A container digest can preserve substantial software context, but it should be accompanied by host hardware and driver information when those conditions matter to the claim.

For embodiment and control, the manifest should bind robot and gripper assets to the action decoder. The decoder should identify units, coordinate conventions, target semantics, normalization, gains, frequency, and relevant wrappers. When a primitive executes multiple internal steps, its stopping rule and time budget should be reported. This makes a benchmark interaction count interpretable without assuming it represents a uniform physical duration.

For randomness and tasks, the manifest should record what each generator controls. Training runs, evaluation episodes, scene reconfiguration, and data subsampling may require separate streams or explicit mappings. Initial-state files should be versioned alongside their task definitions and episode indices. The report should identify both the sampled population and the procedure that turns a seed or state record into an initial condition.

For evaluation, the manifest should preserve the exact evaluator and its inputs. A reported success rate needs the denominator, task weighting, failure treatment, horizon, and checkpoint-selection procedure. Raw per-episode outcomes support alternative aggregation and uncertainty analysis. Keeping these records separate from a single summary score makes the reported claim easier to examine and reduces ambiguity during later reuse.

Table 5. Proposed reporting objects for a claim-linked configuration manifest.

Reporting object	Recommended contents	Question a reviewer can inspect
Software and assets	Commit/submodules, resolved packages, binary/build IDs, asset checksums, platform	Can the exact source and model population be identified?
Embodiment and action	Robot/gripper description, action decoder, units/frames, gains, wrappers and timing	Can one policy output be interpreted unambiguously?
Randomness and reset	Generator names, stream mapping, per-run/per-episode seeds, states and reset ordering	Can the intended initial-condition distribution be reconstructed?
Task and observations	Task/split IDs, asset population, camera configuration, preprocessing and rewards	Is the evaluated problem the same as the described problem?
Evaluation and provenance	Evaluator commit, horizon, success/termination, checkpoint rule, episode outcomes and result links	Does the configuration support the quantity claimed in the manuscript?

This proposal adapts general reproducibility principles to the dependencies visible in the reviewed artifacts. Its effect on reproduction effort has not been empirically evaluated.

5.2 A staged review before expensive reruns

Figure 2 outlines a staged workflow. First, identify the claim and reproduction target: reconstructing an evaluation condition differs from reproducing a training comparison. Second, resolve the relevant source, dependencies, assets, and configuration. Third, inspect whether the evaluator and its output definition match the reported quantity. Only then should execution evidence be requested at a scale appropriate to the claim.

This review performs the documentation and source-inspection stages only. A future execution stage could begin with environment construction, one reset, and one decoded action before expanding to checkpoint evaluation or retraining. Such checks would establish limited executability, not automatically reproduce the paper's main conclusion. Their results should be reported explicitly rather than inferred from successful installation.

The staging also supports proportionate reviewer requests. A review article describing public configurations does not require retraining every policy to support a claim about documented defaults. A paper claiming replicated performance requires evidence beyond configuration inspection. Matching the requested evidence to the claim avoids both unsupported certification and unnecessary computational burden.

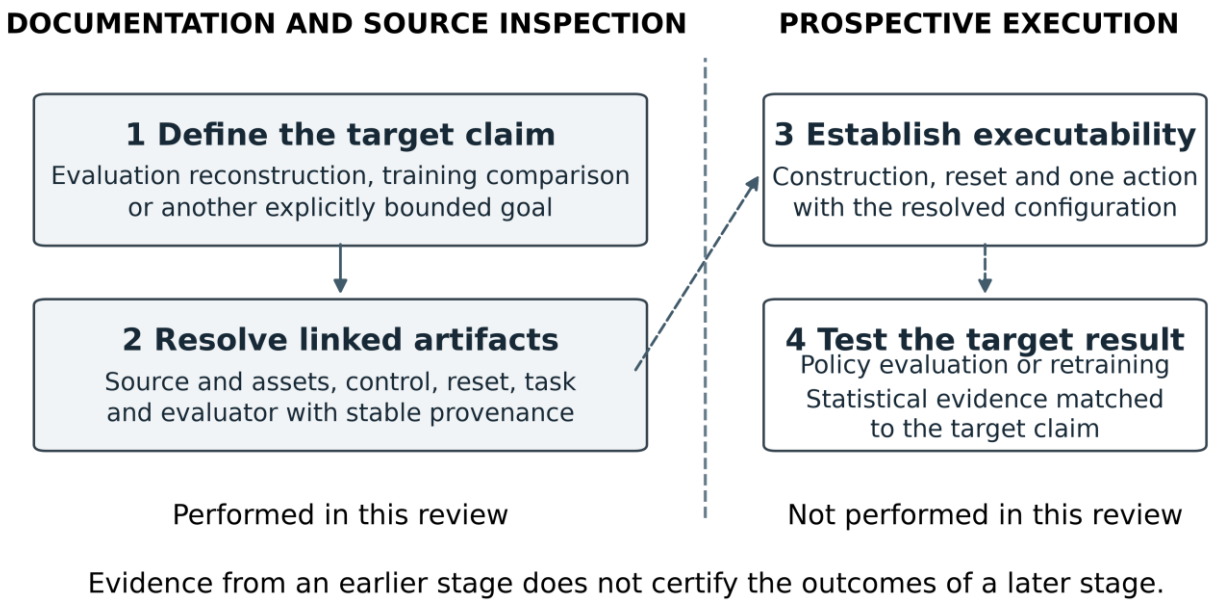


Figure 2. Staged assessment of reproduction conditions. Documentation and source inspection are the stages performed in this review. Establishing executability and testing a target result require additional evidence. Passing an earlier stage does not certify a later stage.

6 Discussion

6.1 What the comparison adds to existing reviews

The central contribution is the connection between named reporting fields and the artifacts needed to interpret them together. Existing simulator and manipulation reviews explain which tools and methods are available. Reproducibility programs explain why artifacts and evaluation procedures should be shared. The present synthesis adds source-level examples showing where configuration reconstruction can remain ambiguous despite substantial public information [5,6,19].

The examples favor a dependency interpretation over a simple missing-information explanation. A lower-bound dependency may be deliberate support for a range of installations. A constructor default may be intentionally overridden by a demonstration script. A task-horizon change may improve a benchmark while creating a boundary between evaluation protocols. These observations do not justify attributing poor practice or invalid results to their developers.

The reporting challenge is preserving the relationship between those design choices and a scientific claim. A current installation guide helps new users, while a historical experiment needs a stable record of what was selected. Both purposes can be served when evolving documentation points to release-specific configurations and archived evaluation inputs. A manifest should preserve that distinction rather than replace a project's broader documentation.

6.2 Implications for embodied intelligence and engineering use

Manipulation policies increasingly combine vision, language, action representations, and heterogeneous robot data. Configuration ambiguity can arise at the interfaces between these components, including camera preprocessing, language-to-task mapping, and controller decoding. The cases involving CALVIN, LIBERO, and RoboCasa make these interfaces visible, while robomimic shows how environment metadata can travel with a dataset [13,14,16,17]. The review does not evaluate the robustness of any resulting foundation model.

For engineering reuse, recovering a configuration is also distinct from validating its physical fidelity. A perfectly repeatable simulated contact model may still differ from a physical gripper-object interaction. Domain randomization addresses a related transfer problem, but documenting its parameter distributions remains necessary for understanding what variation a policy encountered [7]. Simulation reporting and physical validation should therefore be connected without being conflated.

The proposed manifest is most useful as a shared index into existing artifacts. It should not become another detached checklist that reports compliance without supplying evidence. Its value depends on whether a reader can follow the links from a claim to a configuration, then to the assets and evaluator that implement it. Whether the manifest reduces ambiguity or effort remains to be evaluated with users or reproduction studies.

6.3 Limitations and the next evidence needed

The purposive sample emphasizes accessible and comparatively mature software. It may underrepresent abandoned projects, proprietary simulators, dexterous-hand benchmarks, deformable manipulation, and work without public repositories. The evidence cannot estimate how frequently the wider literature reports any configuration field. Expanding the corpus with a prospectively specified search and independent extraction would be necessary for such prevalence claims.

Snapshot choice creates another boundary. Current source can clarify today's mechanisms but cannot reconstruct every historical experiment. The historical ManiSkill2 tag and CALVIN submodule inspection reduce specific ambiguities without resolving all release histories. A follow-up study could sample publication-linked releases and compare them with current branches using an explicit migration taxonomy. That design would distinguish documentation evolution from changes in experimental conditions.

The extraction also involves interpretation, particularly where configuration is distributed across wrappers and imported dependencies. A single workflow cannot supply inter-rater reliability. Stable locators and explicit evidence boundaries make judgments inspectable, but they do not replace independent verification. A future audit could preregister field definitions and examine agreement on a held-out set of benchmark artifacts.

Finally, no installation, trajectory replay, policy evaluation, or training reproduction was conducted. Source declarations may differ from realized behavior because of platform constraints, hidden overrides, or defects. The

framework's usefulness could be tested by measuring whether independently supplied manifests reduce unresolved choices or reproduction effort. Such a test should define the outcome in advance and include unsuccessful attempts, rather than report only completed reproductions.

7 Conclusions

Configuration reporting in robot manipulation simulation is most informative when it links an experimental claim to mutually compatible software, embodiment, control, randomness, and evaluation conditions. The ten reviewed entries expose useful mechanisms across all five dimensions, but those mechanisms serve different purposes and evidence levels. Public implementation information should therefore be interpreted through its source, version, and relationship to a selected experiment.

The review provides an evidence table, immutable repository references, and a reporting manifest for inspecting those relationships. Release-specific horizons, entry-point timing overrides, controller dependencies, and data-dependent resets illustrate why isolated field names are insufficient. These are documentation-based conclusions, with no claim of reproduced benchmark performance. Their immediate use is to make reproduction requirements visible before expensive execution begins.

Supplementary Materials

Table S1: Configuration evidence with stable file locators and evidence boundaries; Table S2: Repository snapshot identities; Table S3: Literature evidence and reading record; Table S4: Search and retrieval record; File S1: Reporting manifest template; File S2: Supplementary methods and codebook.

Author Contributions

[To be completed by the authors using the CRediT roles and their actual contributions.]

Funding

[To be completed by the authors with verified funding information or an accurate no-external-funding statement.]

Institutional Review Board Statement

Not applicable. This review examines public technical publications, documentation, and source code and involves no new human or animal research.

Informed Consent Statement

Not applicable.

Data Availability Statement

The literature-derived configuration observations, repository identities, source locators, and reporting template are included in the supplementary materials. The reviewed implementations and publications remain available through the cited third-party sources. No robot experiments or policy-training data were generated for this review.

Acknowledgments

[To be completed by the authors.]

Conflicts of Interest

[To be completed by the authors after checking all relevant relationships and interests.]

References

1. Henderson, P.; Islam, R.; Bachman, P.; Pineau, J.; Precup, D.; Meger, D. Deep Reinforcement Learning That Matters. Proceedings of the AAAI Conference on Artificial Intelligence 2018, 32. https://doi.org/10.1609/aaai.v32i1.11694. Pagination unavailable in retrieved Crossref record; paper text checked in arXiv:1709.06560.
2. Lynnerup, N.A.; Nolling, L.; Hasle, R.; Hallam, J. A Survey on Reproducibility by Evaluating Deep Reinforcement Learning Algorithms on Real-World Robots. Proceedings of the Conference on Robot Learning 2020, 100, 466–489. https://proceedings.mlr.press/v100/lynnerup20a.html.

3. Agarwal, R.; Schwarzer, M.; Castro, P.S.; Courville, A.; Bellemare, M.G. Deep Reinforcement Learning at the Edge of the Statistical Precipice. Advances in Neural Information Processing Systems 2021, 34, 29304–29320. https://papers.nips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html.

4. Patterson, A.; Neumann, S.; White, M.; White, A. Empirical Design in Reinforcement Learning. Journal of Machine Learning Research 2024, 25(318), 1–63. https://jmlr.org/papers/v25/23-0183.html.

5. Wong, L.H.K.; Kang, X.; Bai, K.; Zhang, J. A Survey of Robotic Navigation and Manipulation with Physics Simulators in the Era of Embodied AI. ACM Computing Surveys 2026. https://doi.org/10.1145/3856802. Advance online publication, 3 October 2026; volume and pagination not assigned in the retrieved Crossref record.

6. Acs, M.; Zhong, X. A scoping review of reinforcement learning methods for long horizon robotic manipulation. Discover Robotics 2026, 2, 22. https://doi.org/10.1007/s44430-026-00037-1.

7. Muratore, F.; Ramos, F.; Turk, G.; Yu, W.; Gienger, M.; Peters, J. Robot Learning From Randomized Simulations: A Review. Frontiers in Robotics and AI 2022, 9, 799893. https://doi.org/10.3389/frobt.2022.799893.

8. Zhu, Y.; Wong, J.; Mandlekar, A.; Martín-Martín, R.; Joshi, A.; Lin, K.; Maddukuri, A.; Nasiriany, S.; Zhu, Y. robosuite: A Modular Simulation Framework and Benchmark for Robot Learning. arXiv 2020. arXiv:2009.12293. https://doi.org/10.48550/arXiv.2009.12293. Version retrieved 19 October 2026; latest revision date 2025/01/18

9. James, S.; Ma, Z.; Arrojo, D.R.; Davison, A.J. RLBench: The Robot Learning Benchmark & Learning Environment. IEEE Robotics and Automation Letters 2020, 5, 3019-3026. https://doi.org/10.1109/lra.2020.2974707.

10. Yu, T.; Quillen, D.; He, Z.; Julian, R.; Hausman, K.; Finn, C.; Levine, S. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. Proceedings of the Conference on Robot Learning 2020, 100, 1094–1100. https://proceedings.mlr.press/v100/yu20a.html.

11. Gu, J.; Xiang, F.; Li, X.; Ling, Z.; Liu, X.; Mu, T.; Tang, Y.; Tao, S.; Wei, X.; Yao, Y.; Yuan, X.; Xie, P.; Huang, Z.; Chen, R.; Su, H. ManiSkill2: A Unified Benchmark for Generalizable Manipulation Skills. arXiv 2023. arXiv:2302.04659. https://doi.org/10.48550/arXiv.2302.04659. Version retrieved 19 October 2026; latest revision date 2023/02/09

12. Tao, S.; Xiang, F.; Shukla, A.; Qin, Y.; Hinrichsen, X.; Yuan, X.; Bao, C.; Lin, X.; Liu, Y.; Chan, T.K.; Gao, Y.; Li, X.; Mu, T.; Xiao, N.; Gurha, A.; Rajesh, V.; Choi, Y.; Chen, Y.R.; Huang, Z.; Calandra, R.; Chen, R.; Luo, S.; Su, H. Demonstrating GPU Parallelized Robot Simulation and Rendering for Generalizable Embodied AI with ManiSkill3. Proceedings of Robotics: Science and Systems XXI 2025. https://doi.org/10.15607/rss.2025.xxi.021. Paper 21; source text cross-checked against the authors' accessible arXiv manuscript 2410.00425.

13. Mees, O.; Hermann, L.; Rosete-Beas, E.; Burgard, W. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. IEEE Robotics and Automation Letters 2022, 7, 7327-7334. https://doi.org/10.1109/lra.2022.3180108.

14. Liu, B.; Zhu, Y.; Gao, C.; Feng, Y.; Liu, Q.; Zhu, Y.; Stone, P. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. arXiv 2023. arXiv:2306.03310. https://doi.org/10.48550/arXiv.2306.03310. Version retrieved 19 October 2026; latest revision date 2023/10/14

15. Nasiriany, S.; Maddukuri, A.; Zhang, L.; Parikh, A.; Lo, A.; Joshi, A.; Mandlekar, A.; Zhu, Y. RoboCasa: Large-Scale Simulation of Everyday Tasks for Generalist Robots. arXiv 2024. arXiv:2406.02523. https://doi.org/10.48550/arXiv.2406.02523. Version retrieved 19 October 2026; latest revision date 2024/06/04

16. Nasiriany, S.; Nasiriany, S.; Maddukuri, A.; Zhu, Y. RoboCasa365: A Large-Scale Simulation Framework for Training and Benchmarking Generalist Robots. International Conference on Learning Representations 2026. https://robocasa.ai/assets/robocasa365_iclr26.pdf. Published conference manuscript retrieved from the official project website.

17. Mandlekar, A.; Xu, D.; Wong, J.; Nasiriany, S.; Wang, C.; Kulkarni, R.; Fei-Fei, L.; Savarese, S.; Zhu, Y.; Martín-Martín, R. What Matters in Learning from Offline Human Demonstrations for Robot Manipulation. Proceedings of the 5th Conference on Robot Learning 2022, 164, 1678–1690. https://proceedings.mlr.press/v164/mandlekar22a.html.

18. Zeng, A.; Florence, P.; Tompson, J.; Welker, S.; Chien, J.; Attarian, M.; Armstrong, T.; Krasin, I.; Duong, D.; Sindhwani, V.; Lee, J. Transporter Networks: Rearranging the Visual World for Robotic Manipulation. Proceedings of the 2020 Conference on Robot Learning 2021, 155, 726–747. https://proceedings.mlr.press/v155/zeng21a.html.

19. Pineau, J.; Vincent-Lamarre, P.; Sinha, K.; Lariviere, V.; Beygelzimer, A.; d'Alche-Buc, F.; Fox, E.; Larochelle, H. Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program). Journal of Machine Learning Research 2021, 22(164), 1–20. https://jmlr.org/papers/v22/20-303.html.

20. ACM SIGSIM PADS. Reproducibility and Artifact Evaluation. 2024. https://sigsim.acm.org/conf/pads/2024/blog/artifact-evaluation/. Accessed 19 October 2026; summarizes ACM artifact and results badges.

21. RLBench Contributors. RLBench Source Repository Snapshot. 2026. https://github.com/stepjam/RLBench/tree/02720bba4c73fe02eb75df946b8791b806028a9d. Commit 02720bba4c73fe02eb75df946b8791b806028a9d; accessed 19 October 2026.

22. LIBERO Contributors. LIBERO Source Repository Snapshot. 2026. https://github.com/Lifelong-Robot-Learning/LIBERO/tree/8f1084e3132a39270c3a13ebe37270a43ece2a01. Commit 8f1084e3132a39270c3a13ebe37270a43ece2a01; accessed 19 October 2026.

23. ManiSkill2 v0.5.3 Contributors. ManiSkill2 v0.5.3 Source Repository Snapshot. 2026. https://github.com/haosulab/ManiSkill/tree/493be36121a9dd06071a57172274babe617b789f. Commit 493be36121a9dd06071a57172274babe617b789f; accessed 19 October 2026.

24. robosuite Contributors. robosuite Source Repository Snapshot. 2026. https://github.com/ARISE-Initiative/robosuite/tree/5ce6643f3092639d08f7b0f90ed1c6a84f50552c. Commit 5ce6643f3092639d08f7b0f90ed1c6a84f50552c; accessed 19 October 2026.

25. ManiSkill3 Contributors. ManiSkill3 Source Repository Snapshot. 2026. https://github.com/haosulab/ManiSkill/tree/62ff3a5896b4d5b4cf0ac4c8d79afe600c9404a3. Commit 62ff3a5896b4d5b4cf0ac4c8d79afe600c9404a3; accessed 19 October 2026.

26. Ravens Contributors. Ravens Source Repository Snapshot. 2026. https://github.com/google-research/ravens/tree/4752ffab487ebd387781ad7890aa540ae2780970. Commit 4752ffab487ebd387781ad7890aa540ae2780970; accessed 19 October 2026.

27. Meta-World Contributors. Meta-World Source Repository Snapshot. 2026. https://github.com/Farama-Foundation/Metaworld/tree/c0dafaf1557f31cad3b96ba2b62462f45638f5a5. Commit c0dafaf1557f31cad3b96ba2b62462f45638f5a5; accessed 19 October 2026.

28. RoboCasa Contributors. RoboCasa Source Repository Snapshot. 2026. https://github.com/robocasa/robocasa/tree/b4d6ef26f0da5635406f23890ffaa34764eda66d. Commit b4d6ef26f0da5635406f23890ffaa34764eda66d; accessed 19 October 2026.

29. CALVIN Contributors. CALVIN Source Repository Snapshot. 2026. https://github.com/mees/calvin/tree/fa03f01f19c65920e18cf37398a9ce859274af76. Commit fa03f01f19c65920e18cf37398a9ce859274af76; accessed 19 October 2026. CALVIN environment submodule pinned at 1431a46bd36bde5903fb6345e68b5ccc30def666.

30. robomimic Contributors. robomimic Source Repository Snapshot. 2026. https://github.com/ARISE-Initiative/robomimic/tree/d309eaecc18acf4152a830a895a6984b8ac71b05. Commit d309eaecc18acf4152a830a895a6984b8ac71b05; accessed 19 October 2026.

31. MuJoCo Developers. MuJoCo Documentation Programming Simulation. 2026. https://mujoco.readthedocs.io/en/stable/programming/simulation.html. Accessed 19 October 2026; mutable stable documentation snapshot retained.

32. Farama Foundation. Gymnasium Environment API. 2026. https://gymnasium.farama.org/api/env/. Accessed 19 October 2026; mutable documentation snapshot retained.